

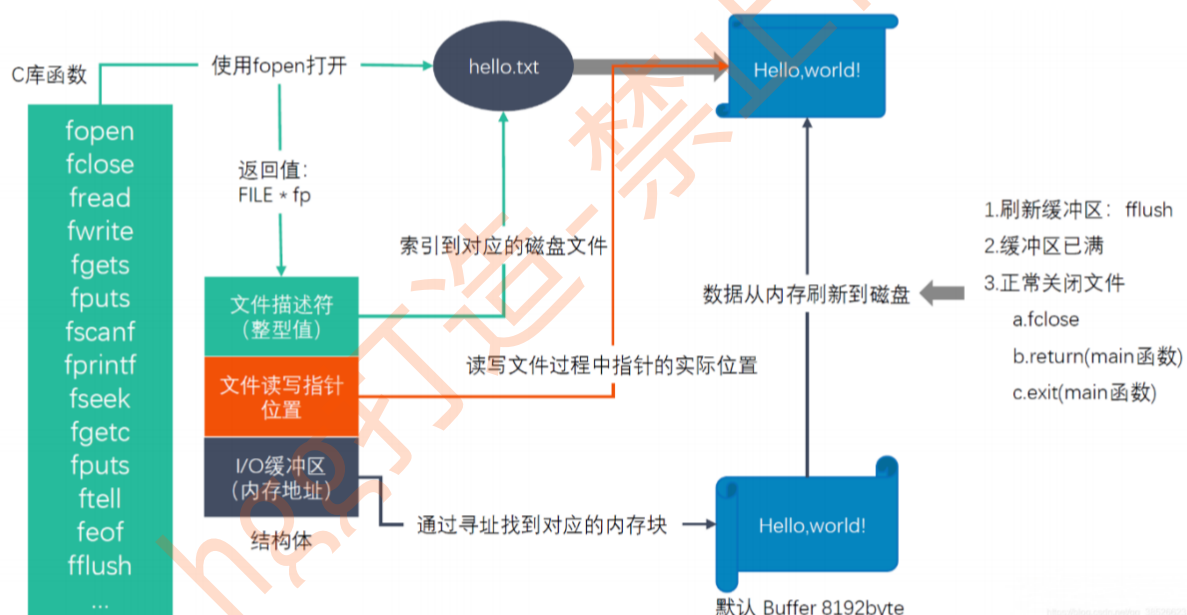
一.c语言标准io

1.理解FILE

```
1 typedef struct
2 {
3     char fd;           //文件描述符 (file definition)
4     unsigned char *buf; //缓冲区
5     size_t bsize;      //缓冲区大小
6     unsigned char *p;  //当前的指向
7     short level;       //缓冲区满或空
8     unsigned short flags; //文件状态标记
9     .....
10
11 }FILE
```

FILE类型是一种结构体类型，在stdio.h中定义，用于存放文件的当前的有关信息。

文件指针: C语言对文件的访问是通过FILE类型指针变量（简称：文件指针）来完成的



2.标准IO的API

a.fopen/fclose

fopen()函数:

原型: `FILE *fopen(char *filename, char *mode);`

功能: 使用mode模式打开指定的filename文件，若打开成功返回一个FILE类型的指针；失败则返回NULL。

参数: filename用来设定打开的文件。mode用来设定要打开的文件类型和指定文件的访问模式，并且必须为字符串格式。

文件的使用方式mode:

使用方式	作用	如果文件不存在
------	----	---------

使用方式	作用	如果文件不存在
"r" (只读)	为了输入数据, 打开一个已经存在的文本文件	出错
"w" (只写)	为了输出数据, 打开一个文本文件	建立一个新的文件
"a" (追加)	向文本文件添加数据	建立一个新的文件
"rb" (只读)	为了输入数据, 打开一个二进制文件	出错
"wb" (只写)	为了输出数据, 打开一个二进制文件	建立一个新的文件
"ab" (追加)	向一个二进制文件尾添加数据	出错
"r+" (读写)	为了读和写, 打开一个文本文件	出错
"w+" (读写)	为了读和写, 建立一个新的文本文件	建立一个新的文件
"a+" (读写)	打开一个文本文件, 在文件尾进行读写	建立一个文件
"rb+" (读写)	为了读和写, 打开一个二进制文件	出错
"wb+" (读写)	为了读和写, 建立一个新的二进制文件	建立一个新的文件
"ab+" (读写)	打开一个二进制文件, 在文件尾进行读写	建立一个新的文件

fopen()函数:

原型: `int fclose(FILE *fp);`

功能: 将文件指针fp所指向的文件关闭。若返回0, 表示关闭成功; 若返回非0, 则表示有错误发生。

示例: `fclose(fp);`

b.fgetc/fputc

fgetc()函数: 读单字符函数

原型: `int fgetc(FILE *fp);`

功能: 读取文件指针fp目前所指文件位置中的字符, 读取完毕, 文件指针自动往下移一个字符位置, 若文件指针已经到文件结尾, 返回-1。

示例: `ch=fgetc(fp);` //从fp所指向的文件中读取一个字符并送入ch。

注意: 调用该函数时, 读取的文件必须是以读或读写的方式打开。

fputc函数(): 写单字符函数

原型: `int fputc(char ch,FILE *fp)`

功能: 把字符ch写入文件指针fp所指向文件的位置, 成功时返回字符的ASCII码, 失败时返回EOF。

c.fgets/fputs

fgets()函数: 读字符串函数

原型: `char *fgets(char *str,int n,FILE *fp);`

功能: 在文件指针fp所指文件位置读取n个字符并放入str字符数组。如果读不到字符串时返回NULL。

fputs()函数: 写字符串函数

原型： `int fputs(char *str, FILE *fp);`

功能：将字符串str写入文件指针fp所指文件的位置。写入数据成功时返回非0值，写入失败时返回EOF。

d.fscanf/fprintf

fscanf()函数：格式化字符串读函数

原型： `int fscanf(FILE *fp, "格式化字符串", 【输入项地址表】);`

功能：从文件指针fp所指向的文件中按照格式字符串指定的格式将文件中的数据送到输入项地址表中。若读取数据成功会返回所读取数据的个数，并将数据按照指定格式存入内存中的变量或数组中，文件指针自动向下移动；若读取失败则返回EOF。

fprintf()函数：格式化字符串写函数

原型： `int fprintf(FILE *fp, "格式化字符串", 【输入项地址表】);`

功能：将输出项表中的变量值按照格式字符串指定的格式输出到文件指针fp所指向的文件位置。

示例：首先初始化要追加的数据，然后以追加写方式打开指定文件。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #define N 3
4  struct stu
5  {
6      char num[20];
7      char name[30];
8      char sex[5];
9  }class[N];
10 int main()
11 {
12     FILE *fp;
13     int i;
14     printf("\n输入该班的数据:\n");
15     fp=fopen("class_list.txt", "wt");
16     if(fp==NULL)
17     {
18         printf("不能打开此文件，按任意键退出!");
19         exit(1);
20     }
21     for(i=0; i<N; i++)
22     {
23         printf("\n第%d个人的信息:\n", i+1);
24         printf("\n学号:");
25         gets(class[i].num);
26         printf("\n姓名:");
27         gets(class[i].name);
28         printf("\n性别:");
29         gets(class[i].sex);
30         fprintf(fp, "%s %s %s\n", class[i].num, class[i].name, class[i].sex);
31     }
32     fclose(fp);
33     fp=fopen("class_list.txt", "rt");
34     printf("该班数据为:\n");
35     printf("学号 姓名 性别\n");
36     i=0;
```

```

37     while(fscanf(fp,"%s %s
%s",class[i].num,class[i].name,class[i].sex)!=EOF)
38     {
39         printf("%s %s %s\n",class[i].num,class[i].name,class[i].sex);
40         i++;
41     }
42     fclose(fp);
43     return 0;
44 }

```

e.fread/write

fread()函数：数据块读函数

原型：int fread(void *buffer,int size,int count,FILE fp);

功能：从文件指针fp所指向的文件的当前位置开始，一次读入size个字节，重复count次，并将读取到的数据存到buffer开始的内存区中，同时将读写位置指针后移sizecount次。该函数的返回值是实际读取的count值。

buffer：是一个指针，在fread函数中，它表示存放读入数据的首地址。在fwrite函数中，它表示要输出的数据在内存中的首地址。

size：表示数据块的字节数。

count：表示要读写的数据块数。

示例：从fp所指的文件中，每次读4个字节（一个实数）送入实型数组fa中，连续读5次，即读5个实数到fa中

fwrite()函数：数据块写函数

原型：int fwrite(void *buffer,int size,int count,FILE fp);

功能：从buffer所指向的内存区开始，一次输出size个字节，重复count次，并将输出到的数据存入fp所指向的文件中，同时将读写位置指针后移sizecount次。

示例：从fa实型数组中，每次读4个字节（一个实数）写入fp所指的文件中，连续读写5次，即读5个实数写到fp所指的文件中。

```

1     struct student_type
2     {
3         char name[20];
4         int num;
5         int age;
6         char addr[30];
7     }stu[40];
8     //一次写1个元素，40次写完
9     for(i=0;i<40;i++) //前提是stu数组中40个元素已经有值存在
10        fwrite(&stu[i],sizeof(struct student_type),1,fp);
11    //一次写40个结构体
12    fwrite(stu,sizeof(struct student_type),40,fp);
13    //一次读1个元素，40次读完
14    for(i=0;i<40;i++) //前提是fp所指向的文件中也有值存在
15        fread(&stu[i],sizeof(struct student_type),1,fp);
16    //一次读取40个结构体
17    fread(stu,sizeof(struct student_type),40,fp);

```

f.rewind

原型: `void rewind(FILE *fp);`

功能: 将文件内部的位置指针移到文件的开始位置。

g.ftell

原型: `int ftell(FILE *fp);`

功能: 得到流式文件的当前位置（相对于文件开头的位移量）。若返回值为-1L表示出错。

h.fseek

原型: `int fseek(FILE *fp, long offset, int whence);`

功能: 文件指针由whence地址移到offset的地址。

whence:

SEEK_SET 文件头

SEEK_CUR 当前位置

SEEK_END 末尾

offset为正数, 这向后偏移; offset为负数, 这向前偏移

i.feof

原型: `int feof(FILE *fp);`

功能: 判断文件指针fp是否处于文件结束的位置。若文件结束, 则返回1, 否则为0。

3.标准C文件操作示例:

a.文件的顺序读写实现内容拷贝

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5
6  #define SIZE 128
7  // FILE *
8  // fgetc fputc
9  void test001()
10 {
11     FILE *fp = NULL;
12     if ((fp = fopen("sources.list", "r")) == NULL)
13     {
14         printf("打开文件失败\n");
15         return;
16     }
17     printf("打开源文件成功\n");
18
19     FILE *target = NULL;
20     if ((target = fopen("target001.list", "w")) == NULL)
21     {
```

```

22     printf("创建文件失败\n");
23     return;
24 }
25 printf("创建目标文件成功\n");
26
27 int count = 0;
28 char text = fgetc(fp);
29 while (text != EOF)
30 {
31     fputc(text, target);
32     count++;
33     text = fgetc(fp);
34 }
35
36 fclose(target);
37 fclose(fp);
38
39 printf("拷贝完成, 复制%d字节\n", count);
40 }
41
42 // fgetc() fputc()
43 void test002()
44 {
45     FILE *fp = NULL;
46     if ((fp = fopen("sources.list", "r")) == NULL)
47     {
48         printf("打开文件失败\n");
49         return;
50     }
51     printf("打开源文件成功\n");
52
53     FILE *target = NULL;
54     if ((target = fopen("target002.list", "w")) == NULL)
55     {
56         printf("创建文件失败\n");
57         return;
58     }
59     printf("创建目标文件成功\n");
60
61     char text[SIZE] = "\0";
62     fgetc(text, SIZE, fp);
63     while (!feof(fp))
64     {
65         fputc(text, target);
66         fgetc(text, SIZE, fp);
67     }
68
69     fclose(target);
70     fclose(fp);
71 }
72
73 struct teacher
74 {
75     char name[SIZE];
76     int salary[12];

```

```

77     int sum;
78 };
79
80 typedef struct teacher teacher;
81
82 // fprintf()   fscanf()
83 void test003()
84 {
85     FILE *fp = NULL;
86
87
88     FILE *target = NULL;
89     if ((target = fopen("target003.list", "w")) == NULL)
90     {
91         printf("创建文件失败\n");
92         return;
93     }
94     printf("创建目标文件成功\n");
95
96     teacher datas[10];
97     // 随机字母初始化姓名(3个大写6个小写),随机数字初始化月薪,sum初始化为0
98     for (int i = 0; i < 10; i++)
99     {
100         memset((void *)datas[i].name, '\0', SIZE);
101         memset((void *)datas[i].name, rand() % 26 + 65, 3);
102         memset((void *)datas[i].name + 3, rand() % 26 + 97, 6);
103
104         for (int j = 0; j < 12; j++)
105         {
106             datas[i].salary[j] = rand() % 5000 + 3000;
107         }
108
109         datas[i].sum = 0;
110     }
111
112     for (int i = 0; i < 10; i++)
113     {
114         int sum = 0;
115         printf("%s ", datas[i].name);
116         for (int j = 0; j < 12; j++)
117         {
118             printf("%d ", datas[i].salary[j]);
119             sum += datas[i].salary[j];
120         }
121         datas[i].sum = sum;
122         printf("%d\n", datas[i].sum);
123     }
124
125     // 写入数据到文件
126     for (int i = 0; i < 10; i++)
127     {
128         fprintf(target, "%s %d %d %d %d %d %d %d %d %d %d %d %d %d\r\n",
129                 datas[i].name,
130                 datas[i].salary[0],
131                 datas[i].salary[1],

```

```

132         datas[i].salary[2],
133         datas[i].salary[3],
134         datas[i].salary[4],
135         datas[i].salary[5],
136         datas[i].salary[6],
137         datas[i].salary[7],
138         datas[i].salary[8],
139         datas[i].salary[9],
140         datas[i].salary[10],
141         datas[i].salary[11],
142         datas[i].sum);
143     fprintf(stderr, "保存%s到文件\n", datas[i].name);
144 }
145
146 fclose(target);
147
148 if ((fp = fopen("target003.list", "r")) == NULL)
149 {
150     printf("打开文件失败\n");
151     return;
152 }
153 printf("打开源文件成功\n");
154
155 printf("从文件读取内容如下:\n");
156 teacher *t = (teacher *)malloc(sizeof(teacher));
157 for (int i = 0; i < 2; i++)
158 {
159     fscanf(fp, "%s %d %d %d %d %d %d %d %d %d %d %d %d %d\n",
160         t->name,
161         t->salary,
162         t->salary + 1,
163         t->salary + 2,
164         t->salary + 3,
165         t->salary + 4,
166         t->salary + 5,
167         t->salary + 6,
168         t->salary + 7,
169         t->salary + 8,
170         t->salary + 9,
171         t->salary + 10,
172         t->salary + 11,
173         &(t->sum));
174
175     fprintf(stdout, "%s %d %d %d %d %d %d %d %d %d %d %d %d %d\n",
176         t->name,
177         t->salary[0],
178         t->salary[1],
179         t->salary[2],
180         t->salary[3],
181         t->salary[4],
182         t->salary[5],
183         t->salary[6],
184         t->salary[7],
185         t->salary[8],
186         t->salary[9],

```

```

187         t->salary[10],
188         t->salary[11],
189         t->sum);
190     }
191
192     free(t);
193     fclose(fp);
194 }
195
196 // fwrite fread
197 void test004()
198 {
199     FILE *fp = NULL;
200
201     FILE *target = NULL;
202     if ((target = fopen("target004.list", "wb")) == NULL)
203     {
204         printf("创建文件失败\n");
205         return;
206     }
207
208     teacher datas[10];
209     // 随机字母初始化姓名(3个大写6个小写),随机数字初始化月薪,sum初始化为0
210     for (int i = 0; i < 10; i++)
211     {
212         memset((void *)datas[i].name, '\0', SIZE);
213         memset((void *)datas[i].name, rand() % 26 + 65, 3);
214         memset((void *)datas[i].name + 3, rand() % 26 + 97, 6);
215
216         for (int j = 0; j < 12; j++)
217         {
218             datas[i].salary[j] = rand() % 5000 + 3000;
219         }
220
221         datas[i].sum = 0;
222     }
223
224     fwrite((void*)datas, sizeof(teacher), 10, target);
225     fclose(target);
226
227     if ((fp = fopen("target004.list", "rb")) == NULL)
228     {
229         printf("打开文件失败\n");
230         return;
231     }
232     printf("打开源文件成功\n");
233
234     teacher t;
235     for(int i=0;i<10;i++)
236     {
237         fread((void*)&t, sizeof(teacher), 1, fp);
238
239         fprintf(stdout, "%s %d %d %d %d %d %d %d %d %d %d %d %d\n",
240             (&t)->name,
241             (&t)->salary[0],

```

```

242         (&t)->salary[1],
243         (&t)->salary[2],
244         (&t)->salary[3],
245         (&t)->salary[4],
246         (&t)->salary[5],
247         (&t)->salary[6],
248         (&t)->salary[7],
249         (&t)->salary[8],
250         (&t)->salary[9],
251         (&t)->salary[10],
252         (&t)->salary[11],
253         (&t)->sum);
254     }
255     fclose(fp);
256 }
257
258 int main()
259 {
260     srand(time(0));
261     puts("+++++++ 1 +++++");
262     test001();
263     puts("+++++++ 2 +++++");
264     test002();
265     puts("+++++++ 3 +++++");
266     test003();
267     puts("+++++++ 4 +++++");
268     test004();
269 }

```

b.文件随机读写，实现数据的随机访问

```

1  //teacher.h
2  #ifndef TEACHER_V251016001_HGG
3  #define TEACHER_V251016001_HGG
4
5  #include <stdio.h>
6  #include <string.h>
7  #include <strings.h>
8  #include <stdlib.h>
9  #include <unistd.h>
10
11 #define NAME_SIZE 128
12 #define SAL_SIZE 12
13 typedef struct Teacher
14 {
15     char name[NAME_SIZE];
16     int sal[SAL_SIZE];
17     int yearSal;
18 }teacher;
19
20 teacher* initTeacher();
21 void printTeacher(teacher* );
22 int getyearSal(teacher* );
23 void freeTeacher(teacher**);
24 #endif

```

```

25
26 //teacher.c
27 #include "teacher.h"
28
29 static char name[27][32] =
30 {
31     {"嬴政"}, {"刘彻"}, {"李世民"}, {"武则天"}, {"成吉思汗"}, \
32     {"康熙"}, {"乾隆"}, {"屈原"}, {"李白"}, {"杜甫"}, \
33     {"苏轼"}, {"曹雪芹"}, {"鲁迅"}, {"孙中山"}, {"屠呦呦"}, \
34     {"张艺谋"}, {"成龙"}, {"莫言"}, {"刘翔"}, {"姚明"}, \
35     {"李娜"}, {"任正非"}, {"马云"}, {"马化腾"}, {"钟南山"}, \
36     {"雷锋"}, {"焦裕禄"}
37 };
38
39 teacher *initTeacher()
40 {
41     teacher *t = (teacher *)malloc(sizeof(teacher));
42     if (t)
43     {
44         bzero((void *)t, sizeof(teacher));
45         strcpy(t->name, name[rand() % 27]);
46         printf("name->%s\n", t->name);
47         for (int i = 0; i < SAL_SIZE; i++)
48         {
49             t->sal[i] = rand() % 3000 + 6000;
50         }
51         t->yearSal = 0;
52     }
53     return t;
54 }
55
56 void printTeacher(teacher *t)
57 {
58     if (t)
59     {
60         printf("%s ", t->name);
61         for (int i = 0; i < SAL_SIZE; i++)
62             printf("%d ", t->sal[i]);
63         printf("%d\n", t->yearSal);
64     }
65 }
66
67 int getyearSal(teacher *t)
68 {
69     if(!t)
70     {
71         return -1;
72     }
73     int sum = 0;
74     for (int i = 0; i < SAL_SIZE; i++)
75         sum+=t->sal[i];
76     t->yearSal=sum;
77     return sum;
78 }
79

```

```

80 void freeTeacher(teacher **t)
81 {
82     if(!t)
83     {
84         return;
85     }
86     if(!(*t))
87     {
88         return;
89     }
90     free(*t);
91     *t=NULL;
92 }
93 //使用标准C对文件随机读写
94 #include <stdio.h>
95 #include <string.h>
96 #include <stdlib.h>
97 #include "teacher.h"
98 #define SIZE 18
99 #define DATA_PATH "data"
100
101 //保存数据到文件
102 void saveTeacher(teacher **,int size);
103
104 int main()
105 {
106     teacher *teacherList[SIZE]={NULL};
107     for(int i=0;i<SIZE;i++)
108     {
109         teacher *t=initTeacher();
110         if(t)
111         {
112             teacherList[i]=t;
113         }
114     }
115
116     for(int i=0;i<SIZE;i++)
117     {
118         printTeacher(teacherList[i]);
119     }
120 }
121
122 //保存数据到文件
123 saveTeacher(teacherList,SIZE);
124
125 FILE *fp=NULL;
126 if(!(fp=fopen(DATA_PATH,"rb+")))
127 {
128     printf("打开数据库失败\n");
129     return 1;
130 }
131
132 //文件数据的随机读取
133 char number[10];
134 teacher temp;

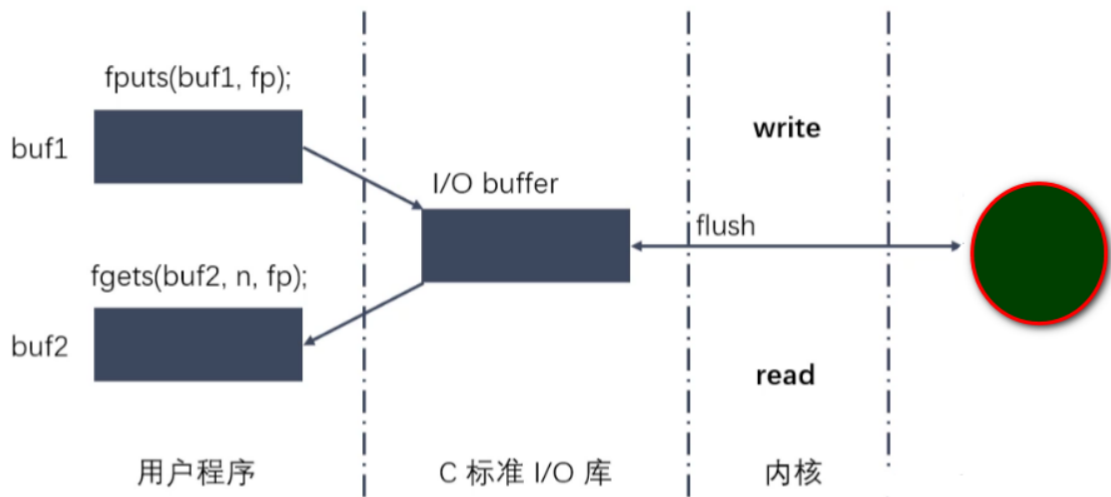
```

```

135     while(1)
136     {
137         printf("请输入要查询的数据编号(0~%d):\n", SIZE-1);
138         scanf("%2s", number);
139         if(atoi(number)>=0&&atoi(number)<=SIZE-1)
140         {
141             if(atoi(number)==0)
142             {
143                 rewind(fp);
144             }
145             else
146             {
147                 //定位到文件, 直接获取数据并输出
148                 fseek(fp, atoi(number)*180, SEEK_SET);
149             }
150             fread((void*)&temp, sizeof(teacher), 1, fp);
151             printTeacher(&temp);
152             continue;
153         }
154         fclose(fp);
155         break;
156     }
157
158
159     for(int i=0; i<SIZE; i++)
160     {
161         freeTeacher(&(teacherList[i]));
162     }
163 }
164
165 //保存数据到文件
166 void saveTeacher(teacher **t, int size)
167 {
168     FILE *fp=NULL;
169     if(!(fp=fopen(DATA_PATH, "wb")))
170     {
171         return;
172     }
173     for(int i=0; i<size; i++)
174     {
175         fwrite(t[i], sizeof(teacher), 1, fp);
176         printf("[%d] 写入完毕, 当前位置:%ld\n", i, ftell(fp));
177     }
178     fclose(fp);
179 }

```

二.Linux系统IO



1.理解文件描述符

在 Linux 中，一切皆文件（everything is a file），包括普通文件、目录、套接字（socket）、管道（pipe）、设备文件等。

文件描述符（File Descriptor, fd）是一个非负整数，是 Linux 为每个打开的文件或资源分配的索引号，用于在内核中标识和操作打开的文件对象。

Linux 启动一个进程时，会默认打开三个文件描述符：

文件描述符	名称	作用
0	stdin	标准输入（键盘）
1	stdout	标准输出（终端）
2	stderr	标准错误输出（终端）

a.文件描述符的分配机制

- 文件描述符是按顺序从最小的可用整数分配的。
- 每个进程都有自己的文件描述符表，是进程私有的。
- 文件描述符表中的每个项指向一个**文件表项**（File Table Entry），其中包含文件偏移量、访问模式等。
- 文件表项再指向**VFS I节点**（与具体文件绑定）。

✅ 这三层结构：文件描述符表 → 文件表项 → inode（VFS）

b.代码检测进程可打开的文件描述符范围

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <time.h>
5 #include <sys/types.h>
6 #include <sys/stat.h>
7 #include <fcntl.h>
8 #include <err.h>
```

```

9  #include <errno.h>
10 #include <unistd.h>
11 int main()
12 {
13     int fd = open("target0044.list", O_RDONLY);
14     if (fd == -1)
15     {
16         perror("");
17         printf("errCode:%d\n", errno);
18     }
19
20     fd = open("target0044.list", O_CREAT | O_WRONLY, 0755);
21     if (fd == -1)
22     {
23         perror("");
24         printf("errCode:%d\n", errno);
25         exit(0);
26     }
27     fprintf(stderr, "创建文件成功 fd=%d\n", fd);
28
29     int count = 1;
30     while (1)
31     {
32         fd = open("target0044.list", O_CREAT | O_WRONLY, 0755);
33         if (fd == -1)
34         {
35             perror("");
36             printf("errCode:%d\n", errno);
37             break;
38         }
39         count++;
40         fprintf(stderr, "创建文件成功 fd=%d\n", fd);
41     }
42
43     fprintf(stderr, "共打开文件%d\n", count);
44
45     pause();
46
47     return 0;
48 }
49
50 /*
51 //open
52 #include <sys/types.h>
53 #include <sys/stat.h>
54 #include <fcntl.h>
55
56 nt open(const char *pathname, int flags);
57 int open(const char *pathname, int flags, mode_t mode);
58
59
60 //close
61 #include <unistd.h>
62 int close(int fd);
63

```

```

64
65 #include <unistd.h>
66
67 int pause(void);
68
69 //测试得，ubuntu2204系统用户会话期支持的文件描述符是0~1023
70
71 */
72

```

2.open/close

```

1 //open
2 #include <sys/types.h>
3 #include <sys/stat.h>
4 #include <fcntl.h>
5
6 int open(const char *pathname, int flags);
7 int open(const char *pathname, int flags, mode_t mode);
8
9
10 //close
11 #include <unistd.h>
12 int close(int fd);

```

功能	打开一个指定的文件并获得文件描述符，或者创建一个新文件	
头文件	#include <sys/types.h> #include <sys/stat.h> #include <fcntl.h>	
原型	int open (const char *pathname, int flags); int open (const char *pathname, int flags, mode_t mode);	
参数	pathname: 即将要打开的文件	
	flags	O_RDONLY: 只读方式打开文件
		O_WRONLY: 只写方式打开文件
		O_RDWR: 读写方式打开文件
		O_CREAT: 如果文件不存在，则创建该文件。
		O_EXCL: 如果使用 O_CREAT 选项且文件存在，则返回错误消息。
		O_NOCTTY: 如果文件为终端，那么终端不可以作为调用 open() 系统调用的那个进程的控制终端。
		O_TRUNC: 如文件已经存在，则删除文件中原有数据。
		O_APPEND: 以追加方式打开文件。
	mode	如果文件被新建，指定其权限为 mode（八进制表示法）
返回值	成功	大于等于 0 的整数（即文件描述符）
	失败	-1
备注	无	

功能	关闭文件并释放相应资源	
头文件	#include <unistd.h>	
原型	int close (int fd);	
参数	fd: 即将要关闭的文件的描述符	
返回值	成功	0
	失败	-1
备注	重复关闭一个已经关闭了的文件或者尚未打开的文件是安全的。	

3.read/write

```
1 #include <unistd.h>
2 ssize_t read(int fd, void *buf, size_t count); // 从fd读取数据
3 ssize_t write(int fd, const void *buf, size_t count); // 向fd写数据
```

4.文件描述符的复制

```
1 int dup(int oldfd); // 复制fd, 返回新的最小可用fd
2 int dup2(int oldfd, int newfd); // 精确复制fd到newfd上
3 int dup3(int oldfd, int newfd, int flags); // 带flag控制
```

4.lseek

```
1 #include <sys/types.h>
2 #include <unistd.h>
3
4 off_t lseek(int fd, off_t offset, int whence);
```

参数说明

fd: 文件描述符, 用于指定要操作的文件。通过open函数打开文件时返回的标识, 代表了一个已打开的文件或设备。

offset: 偏移量, 以字节为单位, 用于指定从参考点(由whence参数指定)开始移动的距离。可以是正数(表示向前移动)、负数(表示向后移动)或零(表示不移动)。

whence: 指定偏移的参考点, 取值及含义如下:

SEEK_SET: 从文件开头开始计算偏移量, 此时offset的值就是新的文件偏移量。

SEEK_CUR: 从当前文件位置开始计算偏移量, 新的文件偏移量为当前位置加上offset的值。

SEEK_END: 从文件末尾开始计算偏移量, 新的文件偏移量为文件大小加上offset的值。

三、函数功能

lseek函数主要用于设置文件的偏移量, 也就是确定下一次对文件进行读写操作时的起始位置。可以实现文件指针的随机定位, 方便对文件中的任意位置进行读写操作, 而不仅仅是顺序读写。

四、返回值

成功时, 返回新的文件偏移量。

出错时, 返回-1, 并设置errno变量以指示错误原因, 常见的错误有:

EBADF: 文件描述符fd无效。

ESPIPE: 文件描述符fd指向的是管道、FIFO 或套接字, 而不是普通文件, 不支持随机访问。

EINVAL: whence参数的值无效, 或者offset的值超出了文件大小的有效范围。

5.系统IO操作实例

a.顺序读写实现文件复制

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <time.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <fcntl.h>
8  #include <err.h>
9  #include <errno.h>
10 #include <unistd.h>
11 #define SIZE 128
12 struct teacher
13 {
14     char name[SIZE];
15     int salary[12];
16     int sum;
17 };
18
19 typedef struct teacher teacher;
20
21 int main()
22 {
23     int fd = open("data", O_WRONLY | O_CREAT, 0644);
24     if (fd == -1)
25     {
26         perror("创建文件失败");
27         printf("errCode:%d\n", errno);
28         return 1;
29     }
30     fprintf(stderr, "创建文件成功");
31
32     teacher datas[10];
33     // 随机字母初始化姓名(3个大写6个小写),随机数字初始化月薪,sum初始化为0
34     for (int i = 0; i < 10; i++)
35     {
36         memset((void *)datas[i].name, '\0', SIZE);
37         memset((void *)datas[i].name, rand() % 26 + 65, 3);
38         memset((void *)datas[i].name + 3, rand() % 26 + 97, 6);
39
40         for (int j = 0; j < 12; j++)
41         {
42             datas[i].salary[j] = rand() % 5000 + 3000;
43         }
44
45         datas[i].sum = 0;
46     }
47
48     for (int i = 0; i < 10; i++)
49     {
50         int sum = 0;
51         printf("%s ", datas[i].name);
52         for (int j = 0; j < 12; j++)
53         {
```

```

54         printf("%d ", datas[i].salary[j]);
55         sum += datas[i].salary[j];
56     }
57     datas[i].sum = sum;
58     printf("%d\n", datas[i].sum);
59 }
60
61 // 写数据
62 size_t writeSize = 0;
63 writeSize = write(fd, datas, sizeof(teacher) * 10);
64 printf("写入数据%d字节\n", writeSize);
65
66 close(fd);
67
68 fd = open("data", O_RDONLY);
69 if (fd == -1)
70 {
71     perror("打开失败");
72     return 2;
73 }
74 fprintf(stderr, "打开成功\n");
75 teacher *t = (teacher *)malloc(sizeof(teacher));
76 if (t)
77 {
78
79     size_t readSize = 0;
80     for (int i = 0; i < 10; i++)
81     {
82         readSize = read(fd, (void *)t, sizeof(teacher));
83         if (readSize == -1)
84         {
85             perror("读数据失败");
86             printf("errCode:%d\n", errno);
87             return 2;
88         }
89         if (readSize == 0)
90         {
91             fprintf(stderr, "文件结束, 读取完毕\n");
92             free(t);
93             close(fd);
94             return 0;
95         }
96
97         fprintf(stderr, "读取数据%d字节\n", readSize);
98         fprintf(stdout, "%s %d %d %d %d %d %d %d %d %d %d %d\n",
99             t->name,
100             t->salary[0],
101             t->salary[1],
102             t->salary[2],
103             t->salary[3],
104             t->salary[4],
105             t->salary[5],
106             t->salary[6],
107             t->salary[7],
108             t->salary[8],

```

```

109         t->salary[9],
110         t->salary[10],
111         t->salary[11],
112         t->sum);
113     }
114 }
115
116     return 0;
117 }
118
119 // #include <unistd.h>
120 // ssize_t write(int fd, const void *buf, size_t count);
121 // ssize_t read(int fd, void *buf, size_t count);

```

b.系统IO随机读取

```

1 // 使用linux系统IO对文件随机读写
2 #include <stdio.h>
3 #include <string.h>
4 #include <stdlib.h>
5 #include <sys/types.h>
6 #include <sys/stat.h>
7 #include <fcntl.h>
8 #include <unistd.h>
9 #include <errno.h>
10 #include <err.h>
11 #include "teacher.h"
12 #define SIZE 18
13 #define DATA_PATH "data"
14 // 保存数据到文件
15 void saveTeacher(teacher **, int size);
16
17 int main()
18 {
19     teacher *teacherList[SIZE] = {NULL};
20     for (int i = 0; i < SIZE; i++)
21     {
22         teacher *t = initTeacher();
23         if (t)
24         {
25             teacherList[i] = t;
26         }
27     }
28
29     for (int i = 0; i < SIZE; i++)
30     {
31         printTeacher(teacherList[i]);
32     }
33     //////////////////////////////////////
34     saveTeacher(teacherList, SIZE);
35     //////////////////////////////////////
36
37     int fd = open(DATA_PATH, O_RDWR);
38     if (fd == -1)
39     {

```

```

40     printf("errCode:%d errMes:", errno);
41     perror("创建文件失败");
42     return 1;
43 }
44 char number[10];
45 teacher temp;
46 while (1)
47 {
48     printf("请输入要查询的数据编号(0~%d):\n", SIZE - 1);
49     scanf("%2s", number);
50     if (atoi(number) >= 0 && atoi(number) <= SIZE - 1)
51     {
52         // 定位到文件, 直接获取数据并输出
53         lseek(fd, atoi(number) * 180, SEEK_SET);
54         read(fd, (void*)&temp, sizeof(teacher));
55         printTeacher(&temp);
56         continue;
57     }
58     close(fd);
59     break;
60 }
61
62 //////////////////////////////////////
63 for (int i = 0; i < SIZE; i++)
64 {
65     freeTeacher(&(teacherList[i]));
66 }
67 }
68
69 // 保存数据到文件
70 void saveTeacher(teacher **t, int size)
71 {
72     int fd = open(DATA_PATH, O_CREAT | O_WRONLY, 0644);
73     if (fd == -1)
74     {
75         printf("errCode:%d errMes:", errno);
76         perror("创建文件失败");
77         return;
78     }
79     printf("新建数据文件成功, fd=%d\n", fd);
80     size_t writeSize = 0;
81     size_t count = 0;
82     for (int i = 0; i < SIZE; i++)
83     {
84         writeSize = write(fd, (void *)t[i], sizeof(teacher));
85         if (writeSize == -1)
86         {
87             printf("errCode:%d errMes:", errno);
88             perror("写文件错误");
89             close(fd);
90             break;
91         }
92         printf("写入数据%d字节\n", writeSize);
93         count += writeSize;
94     }

```

```

95
96     printf("写入数据完成，共写入%d\n", count);
97     close(fd);
98 }

```

三.文件属性

1.命令stat

```

hgg@hgg-vm1014:~$ stat /dev/sda4
文件: /dev/sda4
大小: 0          块: 0          IO 块大小: 4096   块特殊文件
设备: 5h/5d      Inode: 331          硬链接: 1      设备类型: 8,4
权限: (0660/brw-rw----)  Uid: ( 0/   root)  Gid: ( 6/   disk)
访问时间: 2025-10-16 11:04:25.033000093 +0800
修改时间: 2025-10-16 11:04:23.739000066 +0800
变更时间: 2025-10-16 11:04:23.739000066 +0800
创建时间: 2025-10-16 11:04:21.931000029 +0800

```

```

hgg@hgg-vm1014:~$ stat a.c
文件: a.c
大小: 38          块: 8          IO 块大小: 4096   普通文件
设备: 803h/2051d    Inode: 393385      硬链接: 1
权限: (0664/-rw-rw-r--)  Uid: ( 1000/   hgg)  Gid: ( 1000/   hgg)

```

2.函数stat

```

1  #include <sys/types.h>
2  #include <sys/stat.h>
3  #include <unistd.h>
4
5  int stat(const char *pathname, struct stat *statbuf);
6  int fstat(int fd, struct stat *statbuf);
7  int lstat(const char *pathname, struct stat *statbuf);

```

3.理解struct stat

```

1  //man 2 stat
2  struct stat {
3      dev_t    st_dev;        /* ID of device containing file */
4      ino_t    st_ino;        /* Inode number */
5      mode_t    st_mode;      /* File type and mode */
6      nlink_t    st_nlink;    /* Number of hard links */
7      uid_t    st_uid;        /* User ID of owner */
8      gid_t    st_gid;        /* Group ID of owner */
9      dev_t    st_rdev;      /* Device ID (if special file) */
10     off_t    st_size;        /* Total size, in bytes */
11     blksize_t st_blksize;    /* Block size for filesystem I/O */
12     blkcnt_t st_blocks;     /* Number of 512B blocks allocated */
13
14     /* Since Linux 2.6, the kernel supports nanosecond
15     precision for the following timestamp fields.
16     For the details before Linux 2.6, see NOTES. */

```

```

17
18     struct timespec st_atim; /* Time of last access */
19     struct timespec st_mtim; /* Time of last modification */
20     struct timespec st_ctim; /* Time of last status change */
21
22     #define st_atime st_atim.tv_sec /* Backward compatibility */
23     #define st_mtime st_mtim.tv_sec
24     #define st_ctime st_ctim.tv_sec
25
26     //备注: st_mode字段的值是1个6位八进制, 前两位是文件类型, 后4位是文件权限
27 }

```

3.stat应用:分解给定文件的属性

```

1  #include <stdio.h>
2  #include <strings.h>
3  #include <string.h>
4  #include <sys/types.h>
5  #include <sys/stat.h>
6  #include <unistd.h>
7  #include <pwd.h>
8  #include <grp.h>
9  #include <time.h>
10 #define PATH_SIZE 128
11 #define INFO_SIZE 128
12 #define S_IF_MODE 0400
13 #define S_IF_SMODE 04000
14 int main(int argc, char **argv)
15 {
16     if (argc < 2)
17     {
18         printf("缺少参数, 格式为:%s filePath...\n", argv[0]);
19         return 1;
20     }
21     char fileInfo[INFO_SIZE] = "\0";
22     char filePath[PATH_SIZE] = "\0";
23     sprintf(filePath, "%s", argv[1]);
24     printf("Path:%s\n", filePath);
25     struct stat sb;
26     if (stat(filePath, &sb) == -1)
27     {
28         perror("调用stat出错");
29         return 1;
30     }
31
32     // 文件类型
33     char fileType = '\0';
34     switch (sb.st_mode & S_IFMT)
35     {
36     case S_IFSOCK:
37         fileType = 's';
38         break;
39     case S_IFLNK:

```

```

40         fileType = 'l';
41         break;
42     case S_IFREG:
43         fileType = '-';
44         break;
45     case S_IFBLK:
46         fileType = 'b';
47         break;
48     case S_IFCHR:
49         fileType = 'c';
50         break;
51     case S_IFIFO:
52         fileType = 'p';
53         break;
54     case S_IFDIR:
55         fileType = 'd';
56         break;
57     default:
58         fileType = '?';
59         break;
60     }
61
62     // 权限
63     char fileMode[10] = "\0";
64     char ifMode[10] = "rwxrwxrwx";
65     for (int i = 0; i < 9; i++)
66     {
67         fileMode[i] = ((S_IF_MODE >> i) & sb.st_mode) ? ifMode[i] :
        '-';
68     }
69
70     // suid
71     (sb.st_mode & S_IF_SMODE) ? ((fileMode[2] == 'x') ? (fileMode[2] =
        's') : (fileMode[2] = 'S')) : ' ';
72     // sgid
73     (sb.st_mode & (S_IF_SMODE >> 1)) ? ((fileMode[5] == 'x') ?
        (fileMode[5] = 's') : (fileMode[5] = 'S')) : ' ';
74     // stickyBit
75     (sb.st_mode & (S_IF_SMODE >> 2)) ? ((fileMode[8] == 'x') ?
        (fileMode[8] = 't') : (fileMode[8] = 'T')) : ' ';
76
77     // owner:user
78     char ownerUser[32] = "\0";
79     struct passwd *pwd = NULL;
80     pwd = getpwuid(sb.st_uid);
81     if (pwd)
82     {
83         sprintf(ownerUser, "%s", pwd->pw_name);
84     }
85
86     // owner:group
87     char ownerGroup[32] = "\0";
88     struct group *grp = NULL;
89     grp = getgrgid(sb.st_gid);
90     if (grp)

```

```

91     {
92         sprintf(ownerGroup, "%s", grp->gr_name);
93     }
94
95     // 最后修改时间 => 10月 17 12:12
96     struct tm *time = NULL;
97     time = localtime(&(sb.st_mtime));
98     char changeTime[32]="\0";
99     sprintf(changeTime,"%02d月 %02d %02d:%02d",
100         time->tm_mon+1,time->tm_mday,
101         time->tm_hour,time->tm_min);
102
103     sprintf(fileInfo, "%c%s %ld %s %s %ld %s", fileType, fileMode,
104         sb.st_nlink, ownerUser, ownerGroup, sb.st_size,changeTime);
105
106     //////////////////////////////////////
107     // 输出结果
108     fprintf(stdout, "%s %s\n", fileInfo, filePath);
109 }
110
111 // #include <sys/types.h>
112 // #include <pwd.h>
113
114 // struct passwd *getpwnam(const char *name);
115 // struct passwd *getpwuid(uid_t uid);
116 // struct passwd {
117 //     char    *pw_name;        /* username */
118 //     char    *pw_passwd;     /* user password */
119 //     uid_t   pw_uid;         /* user ID */
120 //     gid_t   pw_gid;         /* group ID */
121 //     char    *pw_gecos;      /* user information */
122 //     char    *pw_dir;        /* home directory */
123 //     char    *pw_shell;      /* shell program */
124 // };
125
126 // #include <sys/types.h>
127 // #include <grp.h>
128 // struct group *getgrnam(const char *name);
129 // struct group *getgrgid(gid_t gid);
130
131 // struct group {
132 //     char    *gr_name;        /* group name */
133 //     char    *gr_passwd;     /* group password */
134 //     gid_t   gr_gid;         /* group ID */
135 //     char    **gr_mem;       /* NULL-terminated array of
136 //                             pointers
137 //                             to names of group members */
138 // };
139 // #include <time.h>
140
141 // char *asctime(const struct tm *tm);
142 // char *asctime_r(const struct tm *tm, char *buf);
143

```

```

144 // char *ctime(const time_t *timep);
145 // char *ctime_r(const time_t *timep, char *buf);
146
147 // struct tm *gmtime(const time_t *timep);
148 // struct tm *gmtime_r(const time_t *timep, struct tm *result);
149
150 // struct tm *localtime(const time_t *timep);
151 // struct tm *localtime_r(const time_t *timep, struct tm *result);
152
153 // struct tm
154 // {
155 //     int tm_sec;    /* Seconds (0-60) */
156 //     int tm_min;    /* Minutes (0-59) */
157 //     int tm_hour;   /* Hours (0-23) */
158 //     int tm_mday;   /* Day of the month (1-31) */
159 //     int tm_mon;    /* Month (0-11) */
160 //     int tm_year;   /* Year - 1900 */
161 //     int tm_wday;   /* Day of the week (0-6, Sunday = 0) */
162 //     int tm_yday;   /* Day in the year (0-365, 1 Jan = 0) */
163 //     int tm_isdst;  /* Daylight saving time */
164 // };

```

4.目录操作

a.opendir和readdir

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <strings.h>
4  #include <stdlib.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <unistd.h>
8  #include <dirent.h>
9  int main(int argc, char **argv)
10 {
11     if (argc < 2)
12     {
13         printf("缺少参数 %s path...\n", argv[0]);
14         return 1;
15     }
16     struct stat sb;
17     char path[128] = "\0";
18     sprintf(path, "%s", argv[1]);
19     if (stat(path, &sb))
20     {
21         perror("调用stat错误");
22         return 2;
23     }
24     if (!S_ISDIR(sb.st_mode))
25     {
26         printf(" %s 不是目录\n", path);
27         return 3;
28     }

```

```

29     DIR *dir = opendir(path);
30     if (dir == NULL)
31     {
32         perror("打开目录错误");
33         return 4;
34     }
35     printf("打开目录%s成功\n", path);
36
37     struct dirent *currentDirect = readdir(dir);
38
39     // printf("name:%s ytpе:%#o\n",currentDirect->d_name,currentDirect-
40     >d_type);
41     // currentDirect=readdir(dir);
42     // printf("name:%s ytpе:%#o\n",currentDirect->d_name,currentDirect-
43     >d_type);
44     // currentDirect=readdir(dir);
45     // printf("name:%s ytpе:%#o\n",currentDirect->d_name,currentDirect-
46     >d_type);
47     // while(currentDirect)
48     // {
49     //     printf("name:%s type:%#o\n",currentDirect-
50     >d_name,currentDirect->d_type);
51     //     currentDirect=readdir(dir);
52     // }
53
54     char fileType='\0';
55     while (currentDirect)
56     {
57         switch (currentDirect->d_type)
58         {
59             case 010:
60                 fileType = '-';
61                 break;
62             case 014:
63                 fileType = 's';
64                 break;
65             case 004:
66                 fileType = 'd';
67                 break;
68             case 006:
69                 fileType = 'b';
70                 break;
71             case 002:
72                 fileType = 'c';
73                 break;
74             case 001:
75                 fileType = 'p';
76                 break;
77             default:
78                 fileType = '?';
79                 break;
80         }
81         printf("type:%c name:%s\n",fileType,currentDirect->d_name);

```

```

80         currentDirect=readdir(dir);
81     }
82
83     return 0;
84 }
85
86 // #include <sys/types.h>
87 // #include <dirent.h>
88 // DIR *opendir(const char *name);
89 // DIR *fdopendir(int fd);
90
91 // #include <dirent.h>
92 // struct dirent *readdir(DIR *dirp);
93 // struct dirent
94 // {
95 //     ino_t d_ino;           /* Inode number */
96 //     off_t d_off;          /* Not an offset; see below */
97 //     unsigned short d_reclen; /* Length of this record */
98 //     unsigned char d_type;   /* Type of file; not supported
99 //                             by all filesystem types */
100 //     char d_name[256];      /* Null-terminated filename */
101 // };

```

b.目录操作实例:按类型统计目录下的文件

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <strings.h>
4  #include <stdlib.h>
5  #include <sys/types.h>
6  #include <sys/stat.h>
7  #include <unistd.h>
8  #include <dirent.h>
9  // 统计目录下各种类型的文件个数
10 typedef struct FileCount
11 {
12     char fileType;
13     unsigned count;
14     char comment[32];
15 } fileCount;
16 //用枚举标识符表示文件类型,当做整形用可作为数组下标引用,简洁代码
17 enum fileType
18 {
19     REG = 0,
20     DRT = 1,
21     CHR,
22     LNK,
23     PED,
24     SOK,
25     BLK,
26     NON = 7,
27     HID
28 };
29
30 int main(int argc, char **argv)

```

```

31 {
32     if (argc < 2)
33     {
34         printf("缺少参数 %s path...\n", argv[0]);
35         return 1;
36     }
37     struct stat sb;
38     char path[128] = "\0";
39     sprintf(path, "%s", argv[1]);
40     if (stat(path, &sb))
41     {
42         perror("调用stat错误");
43         return 2;
44     }
45     if (!S_ISDIR(sb.st_mode))
46     {
47         printf(" %s 不是目录\n", path);
48         return 3;
49     }
50     DIR *dir = opendir(path);
51     if (dir == NULL)
52     {
53         perror("打开目录错误");
54         return 4;
55     }
56     printf("打开目录%s成功\n", path);
57     //统计信息数组的文件类型初始化应该和文件类型枚举保持一致
58     fileCount fileInfoList[] = {
59         {'-', 0, "普通文件"},
60         {'d', 0, "目录"},
61         {'c', 0, "字符设备"},
62         {'l', 0, "连接文件"},
63         {'p', 0, "管道文件"},
64         {'s', 0, "套接字"},
65         {'b', 0, "块设备"},
66         {'?', 0, "未知文件"},
67         {' ', 0, "隐藏文件"}
68     };
69
70     for (int i = 0; i < sizeof(fileInfoList)/sizeof(fileInfoList[0]); i++)
71     {
72         fileInfoList[i].count = 0;
73     }
74
75     struct dirent *currentDirect = readdir(dir);
76     char fileType = '\0';
77     while (currentDirect)
78     {
79         switch (currentDirect->d_type)
80         {
81             case 010:
82                 fileType = '-';
83                 fileInfoList[REG].count++;
84                 break;
85             case 014:

```

```

86         fileType = 's';
87         fileInfoList[SOK].count++;
88         break;
89     case 004:
90         fileType = 'd';
91         fileInfoList[DRT].count++;
92         break;
93     case 006:
94         fileType = 'b';
95         fileInfoList[BLK].count++;
96         break;
97     case 002:
98         fileType = 'c';
99         fileInfoList[CHR].count++;
100        break;
101     case 001:
102         fileType = 'p';
103         fileInfoList[PED].count++;
104         break;
105     case 012:
106         fileType = 'l';
107         fileInfoList[LNK].count++;
108         break;
109     default:
110         fileType = '?';
111         fileInfoList[NON].count++;
112         break;
113     }
114     // .
115     // ..
116     // .hgg
117     if(*(currentDirect->d_name)=='.')
118     {
119         fileInfoList[HID].count++;
120     }
121     currentDirect = readdir(dir);
122 }
123
124
125 for(int i=0;i<sizeof(fileInfoList)/sizeof(fileInfoList[0]);i++)
126 {
127     printf("%c 类型是:%s 个数:%d\n",fileInfoList[i].fileType,
128         fileInfoList[i].comment,fileInfoList[i].count
129     );
130 }
131
132 return 0;
133 }
134

```

四.系统IO综合:自定义命令实现ls -l

1.头文件封装fileInfo.h

```
1 ///////////////////////////////////////////////////////////////////
2 //fileInfo.h
3 ///////////////////////////////////////////////////////////////////
4 #ifndef FILE_INFO_20251017_001_VHGG001
5 #define FILE_INFO_20251017_001_VHGG001
6
7 #include <stdio.h>
8 #include <string.h>
9 #include <strings.h>
10 #include <stdlib.h>
11 #include <sys/types.h>
12 #include <sys/stat.h>
13 #include <unistd.h>
14 #include <dirent.h>
15 #include <pwd.h>
16 #include <grp.h>
17 #include <time.h>
18 #define PATH_SIZE 256
19 #define INFO_SIZE 256
20 #define S_IF_MODE 0400
21 #define S_IF_SMODE 04000
22
23 //分析filePath文件的属性到fileInfo,返回fileInfo
24 char *getFileInfo(char *filePath, char *fileInfo);
25 //分析filePath目录下子文件的属性,服用fileInfo保存子文件信息
26 void getDirectSubsFileInfo(char *filePath, char *fileInfo, char* (*info)(char
*, char *));
27 #endif
```

2.功能实现fileInfo.c

```

1 #include "fileinfo.h"
2 char *getFileInfo(char *filePath, char *fileInfo)
3 {
4     #ifdef DEBUG
5         printf("\t\t\t\t\t进入函数%s,在%s的%d行 filePth:%s fileInfo:%s\n",
6             __FUNCTION__,
7             __FILE__, __LINE__, filePath, fileInfo);
8     #endif
9     if (filePath == NULL || fileInfo == NULL)
10    {
11        #ifdef DEBUG
12            printf("\t\t\t\t\t参数错误 函数%s,在%d行 filePath:%p
13            fileInfo:%p\n", __FUNCTION__,
14                __LINE__, filePath, fileInfo);
15        #endif
16        return NULL;
17    }
18    struct stat sb;
19    if (stat(filePath, &sb) == -1)
20    {
21        perror("调用stat出错");
22        return NULL;
23    }
24 }

```

```

22 #ifdef DEBUG
23     printf("\t\t\t\t调用stat成功 在%s:%d行 \n", __FUNCTION__,
24           __LINE__);
25 #endif
26
27     // 文件类型
28     char fileType = '\0';
29     switch (sb.st_mode & S_IFMT)
30     {
31     case S_IFSOCK:
32         fileType = 's';
33         break;
34     case S_IFLNK:
35         fileType = 'l';
36         break;
37     case S_IFREG:
38         fileType = '-';
39         break;
40     case S_IFBLK:
41         fileType = 'b';
42         break;
43     case S_IFCHR:
44         fileType = 'c';
45         break;
46     case S_IFIFO:
47         fileType = 'p';
48         break;
49     case S_IFDIR:
50         fileType = 'd';
51         break;
52     default:
53         fileType = '?';
54         break;
55     }
56
57 #ifdef DEBUG
58     printf("\t\t\t\t处理文件类型完毕 在%s:%d行 \n", __FUNCTION__,
59           __LINE__);
60 #endif
61
62     // 权限
63     char fileMode[10] = "\0";
64     char ifMode[10] = "rwxrwxrwx";
65     for (int i = 0; i < 9; i++)
66     {
67         fileMode[i] = ((S_IF_MODE >> i) & sb.st_mode) ? ifMode[i] :
68         '-';
69     }
70
71     // suid
72     (sb.st_mode & S_IF_SMODE) ? ((fileMode[2] == 'x') ? (fileMode[2] =
73     's') : (fileMode[2] = 'S')) : ' ';
74
75     // sgid
76     (sb.st_mode & (S_IF_SMODE >> 1)) ? ((fileMode[5] == 'x') ?
77     (fileMode[5] = 's') : (fileMode[5] = 'S')) : ' ';

```

```

74         // stickyBit
75         (sb.st_mode & (S_IF_SMODE >> 2)) ? ((fileMode[8] == 'x') ?
(fileMode[8] = 't') : (fileMode[8] = 'T')) : ' ';
76
77         // owner:user
78         char ownerUser[32] = "\0";
79         struct passwd *pwd = NULL;
80         pwd = getpwuid(sb.st_uid);
81         if (pwd)
82         {
83             sprintf(ownerUser, "%s", pwd->pw_name);
84         }
85
86         // owner:group
87         char ownerGroup[32] = "\0";
88         struct group *grp = NULL;
89         grp = getgrgid(sb.st_gid);
90         if (grp)
91         {
92             sprintf(ownerGroup, "%s", grp->gr_name);
93         }
94
95         // 最后修改时间 => 10月 17 12:12
96         struct tm *time = NULL;
97         time = localtime(&(sb.st_mtime));
98         char changeTime[32] = "\0";
99         sprintf(changeTime, "%02d月 %02d %02d:%02d",
100             time->tm_mon + 1, time->tm_mday,
101             time->tm_hour, time->tm_min);
102
103         sprintf(fileInfo, "%c%s %ld %s %s %ld %s", fileType, fileMode,
sb.st_nlink, ownerUser, ownerGroup, sb.st_size, changeTime);
104
105         return fileInfo;
106     }
107
108     void getDirectSubsFileInfo(char *filePath, char *fileInfo, char *(*info)
(char *, char *))
109     {
110         #ifdef DEBUG
111             printf("\t\t\t\t\t进入函数%s,在%s的%d行\n", __FUNCTION__,
__FILE__, __LINE__);
112         #endif
113
114         if (filePath == NULL || fileInfo == NULL || info == NULL)
115         {
116             #ifdef DEBUG
117                 printf("\t\t\t\t\t参数错误 函数%s,在%d行 filePath:%p fileInfo:%p
info:%p\n", __FUNCTION__,
__LINE__, filePath, fileInfo, info);
118             #endif
119
120             return;
121         }
122
123         struct stat sb;

```

```

125     if (stat(filePath, &sb) == -1)
126     {
127         perror("在getDirectSubFileInfo中stat错误");
128         return;
129     }
130     if (S_ISDIR(sb.st_mode))
131     {
132         printf("%s:\n", filePath);
133         DIR *dir = opendir(filePath);
134         if (dir)
135         {
136             //打开非当前目标时,子文件路径不正确的解决方案1:直接切换到具
体目录下
137             //有缺陷:当需要操作多个目录时,当前目录操作完还有切换回原来的
应用程序路径
138             //chdir(filePath);
139
140             //打开非当前目标时,子文件路径不正确的解决方案2:手动拼接用户
输入的路径
141             // ./fileInfo /dev
142             // ./fileInfo /dev/
143             // ./fileInfo /
144
145             // /dev/tty1 /etc
146             char currentPath[INFO_SIZE]="\0";
147             if(filePath[strlen(filePath)-1]!='/')
148             {
149                 sprintf(currentPath,"%s%c",filePath,'/');
150             }
151             else
152             {
153                 sprintf(currentPath,"%s",filePath);
154             }
155
156             struct dirent *d = readdir(dir);
157             char path[INFO_SIZE*2]="\0";
158             while (d)
159             {
160                 //printf("### 分解%s的属性\n\n",d->d_name);
161                 bzero(path,INFO_SIZE*2);
162                 sprintf(path,"%s%s",currentPath,d->d_name);
163                 if (info(path, fileInfo))
164                 {
165                     printf("%s %s\n", fileInfo, d-
>d_name);
166                 }
167                 d = readdir(dir);
168             }
169         }
170
171         printf("\n");
172         return;
173     }
174     printf("在函数%s,%s不是目录\n", __FUNCTION__, filePath);
175 }

```

3.测试getFileInfo封装

```
1 //////////////////////////////////////////////////
2 // 测试getFileInfo的封装
3 //////////////////////////////////////////////////
4 void testFileInfo()
5 {
6     char path[][32]={
7         "/dev/",
8         ".",
9         "test"
10    };
11    char fileInfo[256]="\0";
12    for(int i=0;i<3;i++)
13    {
14        bzero(fileInfo,sizeof(fileInfo));
15        const char *res=getFileInfo(path[i],fileInfo);
16        if(res)
17        {
18            printf("%s %s\n",fileInfo,path[i]);
19        }
20    }
21 }
22
23 int main(int argc,char **argv)
24 {
25     testFileInfo();
26 }
```

4.主函数main

```
1 //////////////////////////////////////////////////
2 //main.c
3 //////////////////////////////////////////////////
4 #include <stdio.h>
5 #include <string.h>
6 #include <strings.h>
7 #include <stdlib.h>
8 #include <sys/types.h>
9 #include <sys/stat.h>
10 #include <unistd.h>
11 #include <dirent.h>
12 #include "fileinfo.h"
13 #define TEXT_SIZE 1024
14 int main(int argc, char **argv)
15 {
16     int usedDStatus = 0;
17     int usedDPos = 0;
18     char fileInfoText[TEXT_SIZE]="\0";
19     for (int i = 1; i < argc; i++)
20     {
21         if (strcmp(argv[i], "-d") == 0)
22         {
23             usedDStatus = 1;
```

```

24         usedDPos = i;
25         break;
26     }
27 }
28 #ifdef DEBUG
29     if (usedDStatus > 0)
30     {
31         printf("您使用了-d,参数位置:%d\n", usedDPos);
32     }
33     else
34     {
35         printf("没有-d\n");
36     }
37 #endif
38
39
40 ///////////////////////////////////////////////////////////////////
41 ///////////////////////////////////////////////////////////////////
42 // ./test2
43 // ./test2 .
44 // ./test test /dev/ /etv
45 // ./test2 test
46
47 // ./test2 -d
48 // ./test2 -d -d -d
49 // ./test -d test
50 // ./test test -d
51 // ./test -d test /dev/ /etv
52 // ./test -d -d /dev
53
54 if(argc==1)
55 {
56     // printf("查看目录 . 内部的子文件属性\n");
57     getDirectSubsFileInfo(".", fileInfoText, getFileInfo);
58     return 0;
59 }
60 // 如果有参数-d,无需关心文件的类型, 直接看属性就行
61 if (usedDStatus)
62 {
63     int dCount = 0;
64     for (int i = 1; i < argc; i++)
65     {
66         if (strcmp(argv[i], "-d") == 0)
67         {
68             dCount++;
69             continue;
70         }
71         //printf("解析 %s的属性\n", argv[i]);
72         bzero(fileInfoText, INFO_SIZE);
73         if(getFileInfo(argv[i], fileInfoText))
74         {
75             printf("%s %s\n", fileInfoText, argv[i]);
76         }
77     }
78 }

```

```

77         if (dCount == argc - 1)
78         {
79             //printf("查看 %s 的属性\n", ".");
80             bzero(fileInfoText, INFO_SIZE);
81             if (getFileInfo(".", fileInfoText))
82             {
83                 printf("%s %s\n", fileInfoText, ".");
84             }
85         }
86
87         return 0;
88     }
89
90
91     // 如果没有 -d,则是目录则看内部子文件属性,非目录则看本身属性
92     struct stat *sb = (struct stat *)malloc(sizeof(struct stat));
93     for (int i = 1; i < argc; i++)
94     {
95         if (stat(argv[i], sb) == -1)
96         {
97             perror("在Main中stat错误");
98             continue;
99         }
100         if (S_ISDIR(sb->st_mode))
101         {
102             //printf("查看目录 %s内部的子文件属性\n", argv[i]);
103
104             getDirectSubsFileInfo(argv[i], fileInfoText, getFileInfo);
105         }
106         else
107         {
108             // printf("查看%s 属性\n", argv[i]);
109             bzero(fileInfoText, INFO_SIZE);
110             if (getFileInfo(argv[i], fileInfoText))
111             {
112                 printf("%s %s\n", fileInfoText, argv[i]);
113             }
114         }
115     }
116     free(sb);
117
118     //////////////////////////////////////
119     //////////////////////////////////////
120     return 0;
121 }

```

5.开发及测试的makefile

```

1  #编译debug版本
2  fileInfoDebug:
3      cc fileInfo.c fileInfo.h main.c -D DEBUG -o fileInfoDebug
4
5  #编译release版本

```

```
6 fileInfo:
7     cc fileInfo.c fileInfo.h main.c -o fileInfo
8
9 #删除debug版本:如果用windows版本的vscode和Linux交叉编译时,因为时间属性不一致导致不会自
   动更新编译,需要手动删除
10 #也可以通过修改文件的时间属性解决该问题
11 removeFileInfoDebug:
12     rm fileInfoDebug
13     touch fileInfo.c
14     touch fileInfo.h
15     touch main.c
16
17 #删除release版本
18 removeFileInfo:
19     rm fileInfo
20     touch fileInfo.c
21     touch fileInfo.h
22     touch main.c
23
24 #测试不带参数-d
25 testNoD:
26     ./fileInfoDebug
27     ./fileInfoDebug .
28     ./fileInfoDebug test /dev/ /etv
29     ./fileInfoDebug test
30
31 #测试带参数-d
32 testUsed:
33     ./fileInfoDebug -d
34     ./fileInfoDebug -d -d -d
35     ./fileInfoDebug -d test
36     ./fileInfoDebug test -d
37     ./fileInfoDebug -d test /dev/ /etv
38     ./fileInfoDebug -d -d /dev
39
40 #测试只分解特定文件信息的调用
41 testSelf:
42     ./fileInfoDebug test /dev/ /etv
43     ./fileInfoDebug test
44     ./fileInfoDebug -d
45     ./fileInfoDebug -d -d -d
46     ./fileInfoDebug -d test
47     ./fileInfoDebug test -d
48     ./fileInfoDebug -d test /dev/ /etv
49
50 #特定文件信息调用和子目录的分解调用
51 testALL:
52     ./fileInfoDebug
53     ./fileInfoDebug .
54     ./fileInfoDebug test /dev/ /etv
55     ./fileInfoDebug test
56     ./fileInfoDebug -d
57     ./fileInfoDebug -d -d -d
58     ./fileInfoDebug -d test
59     ./fileInfoDebug test -d
```

```
60     ./fileInfoDebug -d test /dev/ /etv
61     ./fileInfoDebug -d -d /dev
62
63     #发布到操作系统
64     install:
65         ln -s /home/hgg/cLanguagePrograme/fileInfo/fileInfo /use/bin/fileInfo
66
67     #从系统卸载
68     uninstall:
69         rm /use/bin/fileInfo
```

hgg打造-禁止传播